

Независимый от поставщика контракт ModelClient

Основной план, модуль 01

Краткое содержание

`ModelClient` отделяет код приложения от протокола конкретного поставщика и выполняет ровно одно обращение к LLM. `ModelRequest` описывает намерение приложения, `ModelResponse` — завершённый результат, а `ModelEvent` — одно типизированное событие потокового ответа. Общий контракт должен нормализовать семантику, необходимую вызывающему коду, но одновременно сохранять исходные блоки, идентификаторы, порядок, причины остановки и сведения об использовании. Потеря неизвестного поля опаснее его временного непонимания. Практика закрепляет эту границу immutable-моделями, deterministic fake и проверяемым lifecycle потока; инженерная защита связывает честное отсутствие provider ID с восстановлением состояния Agent после неоднозначного side effect.

Ключевые понятия

- **ModelClient** — асинхронная граница одного обращения: конкретная adapter implementation преобразует request и нормализует response или error. Не выбирает маршрут, не поддерживает цель Agent и не выполняет tool.
- **ModelRequest** — выражает request_id, model, messages, доступные tool и требуемый response format. Не копирует HTTP request одного API и не описывает routing policy.
- **ModelResponse** — хранит упорядоченные результаты завершённого обращения: text, structured data, tool call, refusal и usage. Не является только строкой.
- **ModelEvent** — представляет смысловой шаг потока: start, delta, output completion, usage или terminal response. Частичный JSON ещё нельзя исполнять.
- **ProviderPayload** — хранит точные provider bytes вместе с provider, kind и media type. Неизвестные расширения доступны для replay и аудита, но не получают полномочий Agent.

Основные идеи

Проблема: одинаковая семантика, разные формы

OpenAI Responses возвращает упорядоченные output Items, Anthropic Messages — content blocks и stop_reason, Gemini generateContent — candidates с content.parts, finishReason и usageMetadata. Во всех трёх системах может появиться текст, структурированный результат, tool call, отказ или ограничение длины. Но совпадение смысла не означает совпадения данных.

Если приложение читает каждую форму напрямую, протокол поставщика проникает в Agent loop, тесты и бизнес-правила. Если адаптер оставляет только строку, он теряет `call_id`, `safety-данные`, `citations`, `reasoning Items`, `thoughtSignature`, подробности кэша и новые типы блоков. Нужны одновременно общая форма и `lossless escape hatch` — доступ к исходным данным без переупаковки с потерями.

Один поток выполнения

```
application
  -> ModelRequest
  -> ModelClient
  -> provider adapter -> provider API
  <- raw response или raw stream events
  <- ModelResponse или ModelEvent stream + сохранённые provider data
```

Для обычного вызова адаптер разбирает весь ответ и возвращает `ModelResponse`. Для потока он выдаёт `ModelEvent` в исходном порядке; сборка всех событий должна приводить к тому же смысловому `ModelResponse`, что и непотоковый вызов. Если поток оборвался, частичный результат остаётся наблюдаемым, но не становится успешным завершённым ответом.

Что приводить к общей форме

Общая семантика	Нормализованное представление
Текст	Упорядоченные текстовые блоки, а не только склеенная строка
Структурированный ответ	Исходный JSON-текст, проверенное значение и идентификатор применённой схемы
<code>tool call</code>	Optional provider call ID, имя, <code>parsed/raw-consistent</code> аргументы и порядок; выполнение остаётся снаружи <code>ModelClient</code>
Сведения об использовании	Общие <code>input</code> , <code>output</code> , <code>total</code> , если поставщик их сообщил; отсутствие остаётся отсутствием
Результат генерации	<code>completed</code> , <code>tool_requested</code> , <code>refused</code> , <code>truncated</code> , <code>blocked</code> или <code>incomplete</code>
Поток	Начало, части блоков, завершение блоков, <code>usage</code> и терминальное событие
Ошибка границы	Общая категория с исходной причиной, статусом, <code>request ID</code> и <code>retry_after</code> , если они есть

Строка, похожая на JSON, не становится структурированным ответом сама по себе. Для этого `ModelRequest` должен потребовать схему, а адаптер — проверить результат по этой схеме.

Что сохранять без потерь

- `provider name`, фактические `model/version` и `response/request IDs`;
- исходные блоки в исходном порядке вместе с неизвестными вариантами;

- provider-specific stop reason, status и детали отказа;
- citations, safety ratings и prompt feedback;
- reasoning Items, encrypted content, opaque fingerprints и caller linkage;
- Gemini `thoughtSignature` и порядок частей, необходимый для продолжения;
- детальные usage-поля: `cached`, `cache write`, `reasoning`, `server-tool usage`;
- HTTP status, заголовки `request-id` / `retry-after` и исходное тело ошибки;
- типы и данные неизвестных потоковых событий.

«Без потерь» означает сохранить полученные байты либо структурное дерево без отбрасывания полей. Преобразовать неизвестный блок в `dict`, удалить поля и затем сериализовать обратно — уже не `lossless round trip`.

Примеры

Примеры сокращены до полей, важных для сравнения; это не полные ответы API.

1. OpenAI Responses: `function_call`

```
{
  "id": "resp_oai_1",
  "status": "completed",
  "output": [{
    "type": "function_call", "call_id": "call_1",
    "name": "get_weather", "arguments": "{\"city\":\"Казань\"}"
  }],
  "usage": {"input_tokens": 42, "output_tokens": 18}
}
```

Общая форма получает `tool call` с provider `call_id`, именем и аргументами. Наличие ID зависит от provider; общий контракт не должен выдумывать его. Соответствие `raw` и `parsed arguments` проверяется отдельно от проверки `ToolSpec.input_schema`. Нельзя оставить только SDK helper `output_text`: Responses использует массив типизированных Items, где рядом могут находиться `message`, `reasoning` и другие блоки. Исходный Item сохраняется целиком.

2. Anthropic Messages: отказ как результат

```
{
  "id": "msg_ant_1", "type": "message",
  "content": [{"type": "text", "text": "Не могу выполнить запрос."}],
  "stop_reason": "refusal",
  "stop_details": {"type": "refusal", "category": "policy"},
  "usage": {"input_tokens": 31, "output_tokens": 9}
}
```

Это успешный HTTP-обмен с терминальным результатом `refused`, а не `ModelTransportError`. Общая форма сохраняет видимый текст, но `stop_reason` и `stop_details` нужны для политики fallback и аудита. В потоке эти сведения появляются позднее текста в `message_delta`.

3. Gemini `generateContent`: структурированный текст и метаданные

```
{
  "candidates": [{
    "content": {"role": "model", "parts": [
      {"text": "{\\"umbrella\\":true}", "thoughtSignature": "opaque..."}
    ]},
    "finishReason": "STOP", "safetyRatings": []
  ]},
  "usageMetadata": {"promptTokenCount": 24, "candidatesTokenCount": 7},
  "modelVersion": "provider-version", "responseId": "gem_1"
}
```

После проверки запрошенной схемы JSON можно представить как структурированный результат. `thoughtSignature`, `safetyRatings`, `modelVersion`, `responseId` и исходный `Part` нельзя выбрасывать. Если `promptFeedback.blockReason` присутствует и `candidates` отсутствуют, это нормальный заблокированный результат, а не ошибка разбора массива.

Потоковые различия

- OpenAI Responses выдаёт typed SSE events, например `response.created`, `response.output_text.delta`, `response.function_call_arguments.delta` и `response.completed`.
- Anthropic Messages выдаёт `message_start`, жизненный цикл каждого content block, `message_delta` и `message_stop`; `input_json_delta.partial_json` нельзя разбирать как завершённый JSON до `content_block_stop`.
- Gemini `streamGenerateContent` возвращает последовательность `GenerateContentResponse`; новый Interactions API использует события `step.start`, `step.delta`, `step.stop` и `interaction.completed`.

`ModelEvent` должен выражать общую семантику, но всегда нести исходный тип и данные события. Неизвестный тип передаётся как `unknown provider event`, а не молча игнорируется.

Реализованный механизм

Практика реализована без SDK и live API:

- `contracts.py` задаёт immutable `ModelRequest`, `ModelResponse`, output/event variants, `Usage`, `ProviderPayload`, ошибки и `ModelClient Protocol`;
- `scripted_client.py` потребляет ровно один сценарий на вызов, сверяет `request_id` и `schema_id`, вызывает injected structured validator и не делает hidden retry или `tool execution`;

- `stream_assembly.py` проверяет sequence, provider identity, output lifecycle, delta/final consistency, usage и единственный terminal response;
- `contract_suite.py` фиксирует общий behavioral contract, который позже должен запускаться без изменения assertions для каждой adapter implementation.

Текущий module suite содержит 67 deterministic tests. Они подтверждают deep immutability JSON, различие usage=None и сообщённого нуля, сохранение raw bytes, structured validation, отсутствие hidden retry, cancellation passthrough и точный observed prefix при `ModelStreamInterrupted`. Один `ScriptedModelClient` ещё не доказывает корректность mapping реальных SDK.

Заметки и наблюдения

Классификация ошибок

- **UnsupportedCapability** — требуемая capability отсутствует до request; не повторять на том же маршруте, сохранить requirement.
- **ModelAuthenticationError** — неверный API key или недостаточно прав; не повторять без изменения configuration, сохранить status/request ID.
- **ModelRequestRejected** — некорректный request; не повторять без исправления, сохранить provider error code и body.
- **ModelRateLimited** — rate limit или quota; только ограниченный retry с `retry_after`, quota scope и request ID.
- **ModelUnavailable** — перегрузка или 5xx; ограниченный retry с budget, сохранить status и provider error type.
- **ModelTimeout** — истёк локальный `deadline`; retry создаёт новое обращение и стоимость, поэтому нужна фаза request.
- **ModelTransportError** — network failure; сохранить исходное exception и факт получения bytes.
- **ModelProtocolError** — неизвестная или invalid response shape; обычно не повторять тем же adapter, сохранить raw body/event.
- **ModelStreamInterrupted** — stream оборван после events; сохранить observed `ModelEvent` prefix и признак неоднозначной полноты.
- **CancelledError** — внешняя cancellation; передать без wrapping и не запускать hidden retry.

`refused`, `truncated`, `blocked`, `tool_requested` и остановка по stop sequence — терминальные результаты LLM, а не транспортные исключения. Так вызывающий код может отдельно решать, показать отказ, продолжить после `tool`, увеличить лимит или выбрать другую LLM.

Матрица контрактных тестов

Один набор поведения должен запускаться для каждой adapter implementation и для deterministic fake. Сейчас assertions исполняются только для `ScriptedModelClient`; provider mappings ниже — цель будущих adapter, а не уже пройденный Gate.

Contract oracle	Целевой provider mapping	Проверенный fake case
Текст и порядок блоков сохранены	OpenAI output ; Anthropic content[] ; Gemini parts[]	Упорядоченный ModelResponse , включая UnknownOutput
Structured output проверен запрошенной схемой	OpenAI text format; Anthropic output format; Gemini response schema	Valid/invalid JSON, schema_id и injected validator
tool call сохраняет optional ID, name, raw/parsed args	function_call ; tool_use ; functionCall	Scripted ToolCallOutput , включая call_id=None
tool не исполняется в ModelClient	Любой provider adapter только нормализует proposal	У fake отсутствует executor
Usage и provider details не теряются	usage ; usage ; usageMetadata	None , reported zero и provider-only counters
Refusal/truncation остаются результатами	Provider status/block/finish reason	ResponseOutcome и соответствующий output variant
Stream собирается в тот же semantic response	Typed events; block lifecycle; response chunks	Один scripted response в unary и streaming форме
Частичный JSON не становится tool call	Arguments/input JSON delta	Interrupted prefix без OutputCompleted
Unknown block/event сохраняется	Новый provider type не отбрасывается	Exact kind и bytes
Error identity и cancellation сохраняются	Status, request ID, retry metadata и cause	Typed errors; исходный CancelledError

Контрактные тесты проверяют наблюдаемое поведение, а не классы SDK. Иначе «общий» набор фактически закрепит реализацию первого адаптера.

Инженерная защита: crash после send_email

Provider может корректно вернуть ToolCallOutput(call_id=None) . Это честное отсутствие provider ID, а не повод генерировать значение внутри ModelClient . Для side effect приложение создаёт собственный стабильный operation_id , сохраняет proposal и состояние до вызова tool , а после crash запускает deterministic recovery без нового решения LLM:

```
SENDING -- crash после внешней попытки --> persisted SENDING
restart --> EFFECT_UNKNOWN --> lookup(operation_id)
  FOUND --> SUCCEEDED, повтор запрещён
  TERMINALLY_ABSENT --> SENDING --> retry --> SUCCEEDED
  UNKNOWN --> EFFECT_UNKNOWN, автоматический повтор запрещён
```

`TERMINALLY_ABSENT` означает доказательство, что предыдущая попытка завершена и не была принята. Обычное «письмо пока не найдено» при eventual consistency или незавершённом request — это `UNKNOWN`, иначе lookup сам создаст duplicate. Recovery принадлежит runtime, а не LLM: model proposal не является источником фактического состояния внешнего сервиса.

Fault-эксперимент в `test_ambiguous_email_recovery.py` проверяет все три ветви:

```
uv run --frozen pytest -q \
  lessons/module_01_provider_contract/tests/test_ambiguous_email_recovery.py
```

Наблюдаемый результат — `3 passed: FOUND` не повторяет принятое письмо, `TERMINALLY_ABSENT` разрешает вторую попытку, а `UNKNOWN` оставляет Agent в `EFFECT_UNKNOWN`. Цена решения — durable state, дополнительный lookup, задержка и возможная ручная проверка. Для безвредных уведомлений предметная политика может предпочесть `at-least-once` и принять duplicate.

Типичные ошибки

1. Свести ответ к `text: str` и потерять параллельные блоки, IDs и citations.
2. Считать любой JSON-подобный текст структурированным ответом без запроса и проверки схемы.
3. Исполнять `tool call` в `ModelClient`, смешивая протокол и полномочия.
4. Разбирать аргументы `tool` до терминального события потока.
5. Превращать отсутствие usage в нули и тем самым искажать стоимость.
6. Считать refusal, safety block или token limit сетевой ошибкой.
7. Падать на новом provider block либо молча его отбрасывать.
8. Повторять внутри адаптера без общего retry budget и наблюдаемой причины.

Ограничения этой фазы

Тема содержит typed Python contracts, `ScriptedModelClient`, stream assembler и deterministic tests, но не содержит wire adapter, SSE parser, live API, backpressure experiment, deadline propagation, routing или полный JSON Schema engine. Поля, которых нет у части providers, остаются optional и не получают выдуманные значения. Один fake не доказывает SDK mapping и не закрывает Gate: общий suite ещё должны пройти две реальные adapter implementation.

Recovery-защита — test-only модель причинной границы, а не production subsystem.

Передаваемый application-owned ID не доказывает его генерацию, общий in-memory store — только stand-in для durable transaction, а `TERMINALLY_ABSENT` — строгий oracle, которого обычный sent-folder может не предоставлять. Эксперимент не обещает exactly-once.

Вопросы для повторения

1. Почему `ModelResponse(text=...)` недостаточен даже для ответа, который визуально содержит только текст?
2. Где должен исполняться `tool call` и какую информацию обязан передать `ModelClient`?
3. Почему отсутствие `usage` нельзя нормализовать в нулевые токены?

4. Что должен сделать адаптер с новым неизвестным content block?
5. Как доказать тестом эквивалентность потокового и обычного ответа?
6. Почему `call_id=None` нельзя заменять случайным provider ID и где должен жить `application-owned operation_id`?
7. Почему «не найдено» не разрешает повтор, пока lookup не доказал `TERMINALLY_ABSENT`?

Итоги

Общий контракт должен стабилизировать смысл, а не копировать JSON первого поставщика и не сводить все ответы к строке. `ModelRequest` выражает намерение, `ModelClient` владеет одним обменом, `ModelEvent` сохраняет причинный порядок потока, а `ModelResponse` представляет завершённый результат. Нормализованные поля делают Agent loop независимым от поставщика; сохранённые provider data предотвращают потерю новых возможностей и диагностической информации. Практика показала дополнительную границу: provider correlation ID нельзя подменять application idempotency identity. При неоднозначном side effect состояние восстанавливает deterministic runtime, а не LLM; автоматический retry допустим только по явной предметной политике и проверяемому oracle.

Источники

Внешние API-утверждения проверены по primary documentation 2026-07-31; локальная реализация и tests — 2026-08-01:

- OpenAI Responses: mapping messages to typed Items
- OpenAI Responses: typed streaming events
- OpenAI API specification, `POST /v1/responses` — OpenAPI 2.3.0
- Anthropic Messages API: stop reasons
- Anthropic Messages API: streaming lifecycle
- Anthropic API versioning — `anthropic-version: 2023-06-01`; новые optional fields, enum values и event types могут добавляться
- Gemini `GenerateContentResponse`
- Gemini API versions — Interactions API доступен в stable `v1`; SDK по умолчанию использует `v1beta`
- Gemini migration to Interactions API — `generateContent` считается legacy, но остаётся поддерживаемым
- Основной план: промпт 1.2
- Практика 1.2 и команды запуска
- Общий behavioral contract
- Fault-эксперимент восстановления